Assignment No. 2 – Backend Server Implementation Documentation

1. Introduction

This assignment focuses on building a simple backend server that simulates a restaurant system. The server provides a food catalog with images and handles order submissions from a frontend application. The objective is to demonstrate communication between frontend and backend using HTTP requests.

---

2. Objectives

- To create a backend server that serves a hardcoded menu.
- To include image URLs for each product in the menu.
- To implement a POST endpoint that logs order details.
- To verify frontend-backend communication.
- To prepare a working server with accessible endpoints.

---

3. Technologies Used

- **Node.js** – Runtime environment
- **Express.js** – Web framework for building APIs
- **JavaScript (ES6)** – Programming language
- **REST API** – Communication standard between frontend and backend

---

4. System Overview

The system consists of:

1. **Menu Endpoint (GET)** – Returns a list of food items.
2. **Order Endpoint (POST)** – Receives and logs order data.
3. **Hardcoded Data** – Menu items stored directly inside the server file.

---

5. Implementation Details

5.1 Hardcoded Menu Server (Q1)

A GET endpoint is created to serve a list of food items.

Features:

- Each item includes:
  - Name
  - Price
  - Category

Example Structure:

```
{
  "name": "Pizza",
  "price": 1200,
  "category": "Fast Food"
}
```

Endpoint:

```
GET /menu
```

Functionality:

- When accessed via browser or app, it returns JSON data containing all menu items.

5.2 Visual Catalog with Image Links (Q2)

Each menu item includes an image URL to display product visuals in the frontend.

Example:

```
{
  "name": "Burger",
  "price": 500,
  "category": "Fast Food",
  "image": "https://images.unsplash.com/photo-..."
}
```

Purpose:

- Enhances user interface by displaying product images.
- Eliminates the need for local image storage.

5.3 Order Logger (Q3)

A POST endpoint is implemented to simulate receiving customer orders.

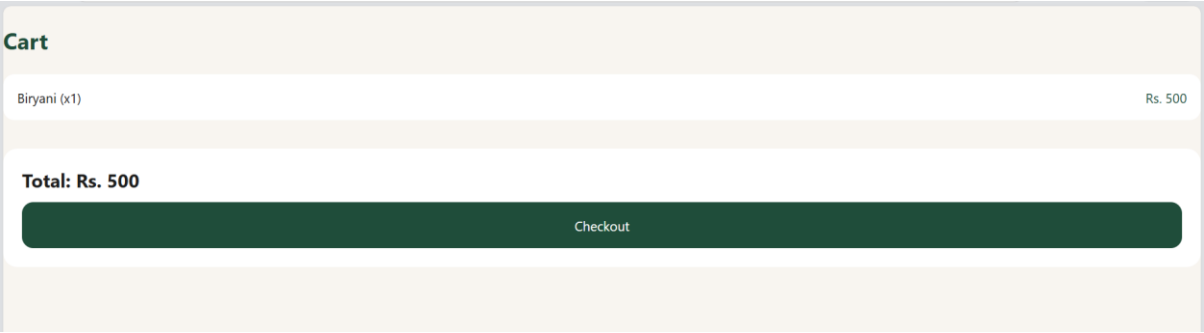
Endpoint:

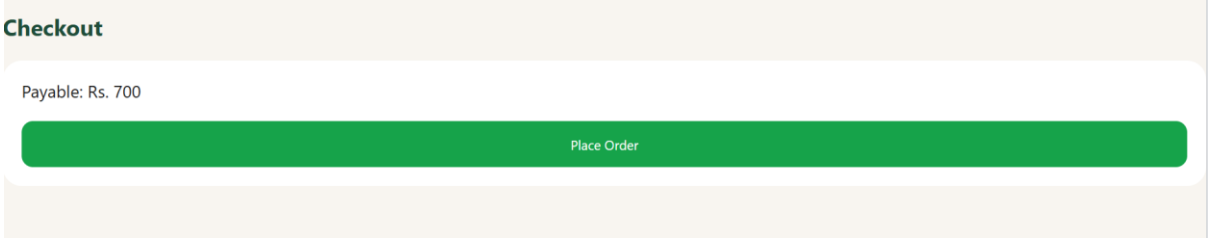
`POST /order`

Functionality:

- Accepts order details from frontend.
- Logs the received order in the server terminal using `console.log()`.

Request Body:





Output in Terminal:

```
Raw Data Received: {
  orderId: 'ORD-5690',
  itemsList: 'Biryani (x1)',
  totalBill: 700,
  deliveryLocation: 'Customer Address'
}

********** NEW ORDER RECEIVED **********
Order ID:  ORD-5690
Items:     Biryani (x1)
Total:     Rs. 700
Address:   Customer Address
****************************************
```

Purpose:

- Confirms successful communication between frontend and backend.

---

6. Server File Structure

The project contains a single file:

```
app.js
```

Contents of app.js:

- Express server setup
- Hardcoded menu data
- GET endpoint (/menu)
- POST endpoint (/order)

---

7. How to Run the Server

Step 1: Install Dependencies

```
npm install express
```

Step 2: Run the Server

```
node server.js
```

Step 3: Access the Menu

Open in browser:

```
http://localhost:3000/menu
```

---

8. Testing the API

GET Request (Menu)

- Open browser or use Postman
- URL: `/menu`
- Returns: JSON menu data

POST Request (Order)

- Use Postman or frontend app
- URL: `/order`
- Method: POST
- Body: JSON format
- Result: Order printed in terminal

README Includes:

- Project description
- Installation steps
- Run commands

---

10. Conclusion

This assignment successfully demonstrates:

- Creation of a REST API using Node.js and Express.
- Serving structured data through endpoints.
- Integration of image URLs for frontend display.
- Handling POST requests and logging data.
- Basic full-stack communication between frontend and backend.

The project provides a foundational understanding of backend development and API design.